ChatGPT

工具调用的正确问题表述：从模式模仿到策略学习（2025–2026 前沿文献深研）

研究问题与形式化基线

过去两年里，工具调用（function calling / tool use）训练的主流范式依然在“**把工具调用当成输出格式或离散标签**”的框架内打转：给定提示词与工具 schema，模型生成一段结构化 JSON，训练信号来自 SFT / DPO 或与“参考答案工具调用”做匹配的规则奖励。其典型形式是把问题写成条件分布学习：

$$p_\theta(y \mid x, \mathcal{T})$$

其中 x 是用户输入与历史对话，$\mathcal{T}$ 是工具集合（schema/描述），y 是包含 tool-call token 的字符串/JSON。像 Hammer 的核心诊断就是：模型在这种设定下会过度依赖“函数名/参数名”的表面模式而不是语义描述，从而造成跨基准不稳定与命名风格迁移失败。[1]

2025–2026 的重要变化是：一批工作开始更明确地把 LLM **当作策略（policy）**，把训练目标表达为最大化期望回报：

$$\max_\theta J(\theta) = \mathbb{E}_{x\sim D,\, y\sim\pi_\theta(\cdot|x)}[R(y|x)]$$

例如 Agent0 在“Preliminaries”里直接将 LLM 表述为策略 $\pi_\thet$，并写出 RL 总目标，同时给出（G）RPO 风格的更新（PPO-style clipped loss + group-based advantage）。[2] ToolRL 也在“RL Training with GRPO”一节明确采用 group-wise advantage + clipped PPO 目标，并强调工具任务的奖励设计需要解决多工具、多参数、细粒度 credit assignment 问题。[3]

但关键在于：**把 LLM 写成 π_θ 并不自动意味着“工具使用被正确地表述为策略学习”**。很多方法仍然把“动作”隐式地塞回 token 空间；奖励也往往仍是“与标注 tool-call 的结构匹配”，而非“在环境里执行工具后达成目标”的真实回报（或其可验证代理）。ToolRL 的 correctness reward 就显式对齐 ground-truth tool calls（工具名、参数名、参数值匹配），并通过最优匹配/归一化把其变成训练奖励。[4]

因此，“正确的问题表述”需要对 **状态、动作、奖励与环境转移** 做出能支撑泛化与决策的建模，而不是把工具 schema 当作模板提示、把工具调用当作输出格式。

在 2025–2026 文献可抽象出一个更贴近本质的基线：把工具调用建模为带外部环境的（部分可观测）序列决策过程。
- 状态（或信息集）s_t：包含用户目标、对话历史、已调用工具及其返回（可能噪声/失败）、以及当前可用工具集合 $\mathcal{T}_t$（工具可能动态检索/增删）。WILDAGTEVAL 明确强调真实世界里 API 执行会噪声/失败、规范会复杂且含 ad-hoc rule，这使得 s_t 的演化与观测噪声变成核心难点。[5]
- 动作 a_t：不是“选一个离散工具ID”，而是结构化对象（可能多 tool calls），例如 ToolRL 的 rollout 成为 $\langle \text{think}, \text{tool_call}, \text{obs}, \text{response}\rangle$ 轨迹。[6]
- 奖励 $R(\tau)$：与“最终任务成功 + 成本/风险/工具正确性 + 过程质量”等多目标相关；TRM 明确指出 outcome-only reward 会造成 gradient conflict（正确中间 tool call 因最终答案错而被惩罚），因此引入逐次工具调用效用的过程奖励。[7]
- 转移 $P(s_{t+1} \mid s_t, a_t)$：由工具执行结果与用户后续输入共同决定；WILDAGTEVAL 在 benchmark 设计中将

execution complexity（信息噪声、feature limitation error、system failure error）系统化，意味着转移中包含不可忽略的随机性/异常路径。[8]

“正确问题表述”的核心，不是把 token 序列当动作，而是明确：**工具调用是对环境施加操作的可组合动作序列**，目标是学习能在变化工具集合、变化命名风格、变化执行噪声下仍稳健的策略。

关键论文与方法的结构化剖析（按五轴）

下面每个条目都沿五个轴解析：问题表述、决策空间、学习信号、泛化机制、以及“为什么仍非真正的工具策略学习”。

Hammer（ICLR 2025 方向：函数遮蔽训练以提升函数调用鲁棒性）

问题表述：本质仍是监督式“生成正确 function-calling 代码/JSON”的学习问题，强调模型目标是输出能完成用户意图的函数调用代码（或空列表）。[9]
可视为

$$p_\theta(y_{\text{call}} \mid x, \mathcal{T})$$

只是在训练分布上做了强扰动（mask）。

决策空间：token 序列空间（输出函数名与参数）。[10]

学习信号：SFT（带标签的正确 tool-call），通过候选列表与标签更新实现“遮蔽后仍对齐标签”。[11]

泛化机制：用随机字符串替换函数名/参数名与默认值，使模型依赖描述而非命名惯例；并通过实验显示未遮蔽模型在测试时 masking 会显著掉点，而 Hammer 掉点更小。[12]
这是一种“工具名不变性（name invariance）”的工程化实现。

仍非真正策略学习的原因：
1) 环境转移被消去：训练目标是“输出正确调用文本”，而非“执行调用、处理噪声/失败、再决策”。WILDAGTEVAL 证明真实 API 执行噪声与规范复杂度会导致 agent failure，但 Hammer 的核心机制并未把这些转移纳入优化目标。[13]
2) 没有显式多步信用分配：遮蔽解决“输入表示”的捷径，但不解决“何时调用/调用后如何反思/重试”的 sequential decision。[14]

ToolRL（2025：面向工具选择与参数化的 reward design + GRPO）

问题表述：明确在 RL 范式下研究“工具选择与应用”任务的奖励设计，并用 GRPO 进行训练；论文强调 reward design 的粒度、尺度、时间动态等对工具学习至关重要。[15]
形式上是 token-policy：

$$\pi_\theta(y \mid x) \text{ 并最大化 } \mathbb{E}[R(y)]$$

其奖励由 format reward + correctness reward 组成。[4]

决策空间：仍主要在 token 序列空间，但通过特殊 token 结构化为“思考/工具调用/回复”，并把 tool_call 解析后插入 obs 形成轨迹。[4]

学习信号：规则奖励（format）+ 基于标注 tool-call 的 correctness reward（工具名/参数名/参数值匹配，甚至构造最优匹配）。 [4]
并采用 group-wise advantage 的 GRPO 更新与 clipped objective。 [16]

泛化机制：作者强调细粒度 reward 能更好反映真实工具调用复杂性，并报告训练时动态缩放 reward（先促格式后促正确性），以及与 PPO/GRPO 初始化差异等。 [17]

仍非真正策略学习的原因：
1) 奖励仍强绑定“参考 tool-call 文本”：虽然是 RL 训练，但 correctness reward 仍在“与标注调用一致”意义上定义，属于结构化模仿的变体，而不是环境回报的直接优化。 [4]
2) 工具开放集问题未解决：即便 reward 细粒度，若工具集 $\mathcal{T}$ 变化（新工具、重命名、组合依赖变化），策略是否能通过“语义层”迁移仍缺少机制；WILDAGTEVAL 与 TheMCPCompany 都显示真实世界工具/规范复杂度巨大且变化频繁。 [18]

Tool-Zero / GG-GRPO（2025：从 Zero 模型出发的 GRPO + “泛化引导”奖励切换）

问题表述：目标是用扩规模 RL 从“Zero model”训练出工具调用能力，提出从宽松 reward（鼓励探索）平滑切换到严格 AST 校验（强调精确 tool integration），通过 sigmoid 控制过渡。 [19]

决策空间：token 序列（工具调用与思考的结构化输出），严格阶段把生成的调用解析为 AST 并与参考 AST 对齐。 [19]

学习信号：GRPO 目标（给出 token-level importance ratio 与 group advantage），并去掉 KL 正则；奖励包含格式、宽松 overlap、严格 AST match、多工具协作奖励与参数值惩罚等。 [20]

泛化机制：宽松阶段给 partial credit 以鼓励探索，严格阶段确保收敛到可执行且“工具无关的推理”——作者直接用“exploration-exploitation dilemma”来解释该设计。 [19]

仍非真正策略学习的原因：
1) 动作语义仍由参考 AST/标注定义：严格阶段等价于“结构化监督”的奖励化；环境中的执行噪声、规范变化（WILDAGTEVAL 的复杂度维度）并未进入训练回报。 [21]
2) 奖励与“工具集表示学习/不变性”仍弱耦合：即便叫“generalization-guided”，其主要机制是 reward schedule 而非学习可迁移的工具语义表征。 [22]

Tool-R0（2026：零人类数据的自博弈工具学习，Generator–Solver 共同进化）

问题表述：把工具学习写成“自博弈 RL 框架”：同一个 base LLM 分成 Generator 与 Solver；Generator 生成工具调用任务（问题、tool menu、gold tool-call），以设计的奖励推动其生成“处于 Solver 能力前沿”的样本；Solver 在筛选/分桶后的课程上用 GRPO 学习预测 tool calls。 [23]
这相当于把训练分布 $p(q, \mathcal{T}, c^{\backslash *})$ 也纳入策略控制（由 Generator 策略诱导）。 [23]

决策空间：
- Generator：token 序列（生成任务与 gold call），并显式把生成分布写为条件策略 $q \sim \pi_\theta(\cdot \mid s)$。 [24]
- Solver：token 序列（输出 tool calls）。 [25]

学习信号：分解奖励（Generator：格式 rfmt、有效性 rvalid、课程 rcurr；Solver：格式 + 准确率），并给出整套迭代算法（冻结一方训练另一方、再构造课程）。 [23]

同时论文讨论自博弈迭代的饱和现象，并把其解释为“接近 Nash-like equilibrium / knowledge boundary”。[26]

泛化机制：通过自生成课程扩大训练分布，作者宣称可在无人工数据下让弱 base LLM 获得可迁移的 tool-calling 能力，并能作为 mid-training 增强后续监督后训练。[27]

仍非真正策略学习的原因：
1) “环境”仍主要是可验证的 JSON/一致性规则：Generator 的 rvalid 强调 tool menu 与 gold call 内部一致、可解析、值可规范化等，本质仍是语法/结构可验证回路，而不是真实 API 执行反馈（尤其是 WILDAGTEVAL 的 execution noise 与错误路径）。[28]
2) 自博弈解决了“数据与课程”，但对“开放世界工具变化”仍脆弱：Generator 学到的是一种任务分布生成器，不等价于学习一个对任意新工具集合 $\mathcal{T}$ 都可迁移的语义 grounding 机制。[29]

TRM（ICLR 2026：Tool-call Reward Model + turn-level advantage，用于解决 gradient conflict 与 reward hacking）

问题表述：指出 outcome-only reward 在工具轨迹上会出现 credit assignment 问题与梯度冲突，于是提出 Tool-call Reward Model（TRM）给每次工具调用评估效用，并把“turn-level reward”与最终 outcome reward 组合。[7]
其训练将 TRM 视为对每个 tool call 的二分类/打分器：用 transformer backbone + binary head，BCE loss：[30]

决策空间：
- 策略仍是 token-level policy optimization（PPO/GRPO），但奖励定义在“turn”层级，并需要把 reward 映射回 token（把回报分配到每次 tool call 的最后一个 token）。[31]
- 关键改动是：在 GRPO 中引入“turn-level advantage estimation”：每一轮工具调用的奖励在组内**独立归一化**，再折扣聚合为 token-level advantage。[32]

学习信号：
- TRM 的监督来自蒸馏/构造的数据（论文描述 workflow），TRM 学到的是“工具调用是否有用”的判别式奖励。[7]
- 策略更新用 PPO/GRPO clipped objective，但 advantage 来自 turn-level reward + outcome reward 的组合，并显式指出 naive group-level advantage 会导致 reward hacking（少用工具以规避惩罚），因此需要 turn-level 处理。[33]

泛化机制：论文声称 TRM 能提升工具调用泛化，允许适配 unseen tools，并强调构造 TRM 的数据量与模型规模影响。[34]

仍非真正策略学习的原因：
1) TRM 仍是“可学习的中间奖励器”，而非环境回报：若工具执行存在噪声/失败/未文档化规则，TRM 的判别是否仍可靠并不保证；WILDAGTEVAL 说明现实复杂度来自规范与执行两端。[35]
2) 仍在 token-policy 上做优化：turn-level advantage 缓解了粒度不匹配，但没有改变“动作空间＝文本 token”的根本错配；它更像是对 credit assignment 的重要修补，而非“正确动作建模”。[36]

DTDR（2026：动态工具依赖检索，把工具子图作为随轨迹演化的上下文）

问题表述：把工具检索写成随计划进展变化的函数 $\omega(q, f_{0:t-1})$，目标是从大工具集里检索最小的、与当前 query 和历史调用相关的依赖子图，从而减少无关工具诱导与 context 长度。[37]

决策空间：检索决策不在 token 里，而是在“候选工具集合/子图”空间；DTDR-C 用“聚类 + 图遍历”把 query 映射到一个依赖图，再基于历史遍历预测下一步工具集合。[38]
DTDR-L 则是冻结 embedding model + 线性层分类器，学习 $P_\phi(f \mid q, f_{0:t-1})$ 并阈值化取子集。[39]

学习信号：来自 demonstrations（函数调用轨迹），训练的是“下一个工具集合预测/对齐”，本质是监督式或弱监督式检索组件学习。[40]

泛化机制：引入 history-aware retrieval，实证表明无 query/history awareness 的方法不足以解决检索任务，并强调 tool dependency（参数来自前序输出）在数据集中显著存在。[41]

仍非真正策略学习的原因：
1) 这是“状态表示/动作可行集构造”的子模块，而非端到端策略：ω 影响 prompt，但不直接在 RL 目标下联合优化（类似 ARISE 批评的“检索与 policy gradient 解耦”的问题）。[42]
2) 工具调用后的适应仍依赖主 agent：检索减少迷惑项，但不会自动学会在执行噪声/错误下自修正。[43]

WILDAGTEVAL（2026：把真实 API 复杂度系统化为评测维度）

问题表述：不是训练算法，而是指出现有基准对 API 世界的理想化假设导致能力评估过于乐观，提出在固定 API system 中注入现实复杂度，并以此评价 agent under real-world API complexity。[44]

决策空间：评测强调 agent 必须在“复杂规范 + 执行噪声/错误”下进行多轮调用与适应；提示词结构要求 agent 基于 prior calls 与 execution results 预测下一次 API call。[45]

学习信号：N/A（benchmark），但它明确了“真实奖励/成功”与“规范遵循”之间的张力：例如参数必须满足 domain-specific 格式规则、执行可能返回 sponsored content 或 cryptic error。[13]

泛化机制：通过覆盖 specification complexity 与 execution complexity 的 8 类复杂度，并支持大量配置组合来评测鲁棒性。[46]

仍非真正策略学习的原因（对领域的启示）：它反而揭示了：若训练只对齐“干净标注 + 理想执行”，策略在真实复杂度下会出现系统性失败模式，因此“正确问题表述”必须把这些复杂度纳入训练目标或可验证代理。[47]

TheMCPCompany（2025/ICLR 2026：大规模任务专用工具集合 + 检索式工具发现）

问题表述：强调随着 MCP 类协议出现，任务专用工具数量激增，提出用大规模 task-specific tool sets 构建通用 agent 的可行性，并在包含超过 18,000 工具的环境中评测；同时比较 browser 工具 vs oracle tool set vs 检索式 MCPAgent。[48]

决策空间：核心决策扩展为两层：先发现/检索工具，再进行调用与多步任务执行；论文的表格显示 oracle tool set 显著优于检索式 MCPAgent，而 MCPAgent 又常优于 browser。[49]

学习信号：主要是系统/评测与工具检索工程（非核心 RL 训练算法）；论文用成本、步数等指标展示工具接口的效率收益。 [49]

泛化机制：真实 REST API 转工具、环境复杂度高、需要检索；结论之一是：即使检索不完美，task-specific tools 仍可提升性能与降低成本，但工具发现仍是瓶颈（oracle 与非 oracle 有明显 gap）。 [49]

仍非真正策略学习的原因：它更像揭示“开放集工具发现 + 调用”这一结构性难题：当 $|\mathcal{T}|$ 巨大时，把工具当离散标签分类几乎必然失败，必须学习“可迁移的工具表示与 grounding”；但论文并未给出把检索与策略优化统一起来的学习型表述。 [50]

AgentPRM（2025：把过程奖励模型用于 agent 场景的 actor–critic 视角）

问题表述：指出 agent 场景的两点困难：多步决策与稀疏回报；提出 AgentPRM 用 Monte Carlo rollouts 自动标注过程奖励目标，迭代训练 PRM 与 policy，并强调其与标准 RLHF pipeline 的兼容性。 [51]

决策空间：依然是策略 $\pi(a \mid s)$（论文图示直接写出 policy 与 PRM 的 actor–critic 结构），强调 PRM 像 critic / value function 一样提供密集监督。 [52]

学习信号：三阶段迭代：rollout 计算 PRM target、监督训练 PRM、再用 RL 更新 policy；并提出 InversePRM 从 demonstrations 学过程奖励。 [53]

泛化机制：理论上通过过程奖励缓解稀疏回报、提高 sample efficiency；并讨论探索、reward shaping、model-predictive reasoning 等挑战。 [51]

仍非真正工具策略学习的原因：
1) PRM 的泛化与正确性是新的瓶颈：工具世界的奖励不只是“推理步骤是否好”，还涉及执行噪声、规范约束与外部状态变化；这在 WILDAGTEVAL 中被系统化。 [54]
2) PRM/actor-critic 仍可能把“工具语义”折叠成对当前工具集合的模式拟合：如果没有 tool-invariant 表示与 grounding，PRM 可能学到的是工具名/参数模式的捷径。 [55]

CoLA（ICML 2025：用潜在动作层重构 LLM 的动作空间）

问题表述：明确指出“LLM 的 RL 训练缺少合适的动作空间结构”，提出学习紧凑潜在动作空间以提升可控性与探索；框架包含 inverse dynamics 提取 latent actions、把 LLM 微调成可接收 latent action 的语言 world model，并训练 policy 产生 latent actions（可用 BC 或 RL）。 [56]

决策空间：从 token 空间转向**潜在连续/离散动作** z，并引入三模块：
- $q_\psi(z \mid \text{future tokens})$ 的逆动力学式提取；
- language world model $p_\theta(x_{t+1} \mid x_{\le t}, z_t)$；
- latent policy $\pi_\phi(z_t \mid x_{\le t})$。 [56]
这显式改变“动作居住空间”。

学习信号：BC 模仿 LM 或 RL 最大化下游奖励；并报告在数学与 agent-based tasks 上相对 token-level RL 的收益。 [56]

泛化机制：潜在动作作为语义压缩，理论上能减少 token-level 动作空间的组合爆炸并提升探索效率。 [57]

仍非真正工具策略学习的原因：CoLA 解决的是“语言生成的动作空间结构”，但工具调用还需要把潜在意图对齐到外部工具集合 $\mathcal{T}$ 并满足 schema；CoLA 本身没有把“新工具/重命名工具/工具执行噪声”作为泛化对象来设计 grounding。 [58]

LSPO（ICML 2025：把自由语言动作抽象为有限潜在策略空间，再用 CFR 优化）

问题表述：在 Werewolf 语言博弈里，作者认为自由语言动作空间巨大但“底层策略空间相对紧凑”，于是构造有限 latent strategy space，把原问题重写为抽象博弈，再用 CFR/Deep CFR 求策略，最后用 DPO 把 LLM 对齐到学到的策略（由 regret 生成偏好数据）。 [59]

决策空间：关键决策发生在离散 latent strategy z 上，而语言 utterance 作为 z 的实例化；论文用 embedding + k-means 将话术聚类为 latent action。 [60]

学习信号：博弈求解（CFR）产生“更优策略”，再用 DPO 做偏好对齐；偏好标签由 regret 值决定（低 regret 更偏好）。 [61]

泛化机制：通过迭代扩展 latent space，提高探索；“策略层”优化减轻语言分布偏置。 [62]

仍非真正工具策略学习的原因：LSPO 的 latent space 是在固定游戏环境与特定语用里通过聚类构造的；工具世界要求把 latent intent 与可变工具集合 $\mathcal{T}$ 做可组合的 grounding，并处理执行噪声与规范约束；LSPO 提供了“潜在策略层”的重要范式，但缺少工具语义对齐与可执行性约束的统一建模。 [63]

GSPO / SAPO / Scaf-GRPO / RLVR（2025–2026：LLM-RL 的稳定性、粒度与“学习悬崖”）

这些工作并非专为工具调用，但它们决定了“如果我们把工具使用当策略学习”，那么 **策略优化算法与奖励粒度该如何匹配**。

GSPO（2025）
- **问题表述**：指出 GRPO 的 token-level importance sampling 在长序列/MoE 下 ill-posed，提出序列级 importance ratio：

$$\rho_{\text{seq}} = \left(\frac{\pi_\theta(y \mid x)}{\pi_{\text{old}}(y \mid x)} \right)^{1/|y|}$$

并做 sequence-level clipping 与优化。 [64]
- **决策空间**：仍是序列策略，但优化单元从 token 转向“序列”，强调“优化单元应匹配奖励单元”。 [64]
- **仍非工具策略学习**：它解决“怎么更稳定地更新生成策略”，但不决定动作是否应是 token/工具调用对象；也不解决开放工具集合与执行转移。 [65]

SAPO（2025）
- **问题表述**：针对 GRPO/GSPO 的 hard clipping，提出温度控制的 soft gate，实现 token-adaptive 且序列一致的连续 trust region，并在预备章节明确写出 GRPO 与 GSPO 目标，再给出 SAPO 目标与梯度权重。 [66]
- **仍非工具策略学习**：同 GSPO，属于“优化器层”改进；工具调用需要动作建模与 grounding。 [67]

Scaf-GRPO（ICLR 2026）
- **问题表述**：指出 RLVR/GRPO 存在“learning cliff”：难题长期零奖励导致 advantage 消失；提出当学习停滞

时注入分层提示（knowledge/planning/solution）并用确定性搜索找到最小有效提示，替换失败轨迹以恢复学习信号。 [68]
- **仍非工具策略学习**：它解决“零奖励不可见”的优化问题，但提示注入是一种外部 scaffold；工具策略学习更关心在真实执行反馈下的自修正与探索。 [69]

RLVR 机制分析（ICLR 2026）
- **问题表述**：研究“仅凭答案正确性这种可验证回报，为什么 RLVR 仍能激励正确推理”，提出理论框架解释其隐式激励机制，并提出 CoT-Pass@K 指标。 [70]
- **启示**：对工具世界来说，这支持一个关键观点：即便终局 reward 稀疏，只要 base policy 已有足够先验，RL 也可能通过改变路径概率来提升“正确过程”。但工具世界的执行噪声与规范复杂度意味着“可验证回报”的设计比纯推理更难，TRM/AgentPRM 等正是在补这一环。 [71]

GoRL / FPO / AEPO（2025–2026：生成式策略为“动作分布建模”提供结构原则）

这些工作主要在连续控制领域，但它们给出一个对工具调用极有价值的结构原则：**把“可优化性”与“表达能力/生成过程”解耦**。

- **GoRL（2025）**：明确提出“decoupling optimization from generation”，优化一个可 tractable 的 latent policy，再用条件生成 decoder 合成动作，并用两时间尺度更新保持稳定同时逐步提升表达力。 [72]
- **FPO（ICLR 2026）**：把 flow matching 引入 policy gradient，用 flow matching loss 构造近似 likelihood ratio，并在 PPO-clip 框架下做 advantage-weighted 更新：

  $$\hat{r}_{\mathrm{FPO}}(\theta) = \exp(\hat{L}_{\mathrm{CFM},\theta_{\mathrm{old}}} - \hat{L}_{\mathrm{CFM},\theta})$$

  并最大化带 clip 的 surrogate objective。 [73]
- **AEPO（NeurIPS 2025）**：在 diffusion-guided 离线 RL 中给出解析/闭式的中间指导估计思路，强调 energy-guidance 的可计算性问题。 [74]

对工具调用的直接启示：如果把工具调用动作当作“可执行的结构对象”，我们可能需要一种类似 GoRL 的两层表示：上层策略在“语义/意图的可优化空间”里学习（比如 latent action z 或“操作原语”），下层生成器把它映射到满足 schema、满足约束并能与环境交互的具体 tool-call（包含参数与多步依赖）。 [75]

跨论文对比与共同失配：工具使用到底“被建模成了什么”

把 2025–2026 的代表性方法放在一起，会发现它们在“表面上都在做 RL/agent”，但**实际建模对象差异巨大**：

- **分类/匹配器视角**：动作空间被压成“工具ID或候选函数中的一个”；训练本质是 $P(f \mid x)$ 的分类或排序，哪怕输出形式是 JSON。Hammer 的核心问题诊断就是这种设定会诱导模型记忆命名模式。 [1]
- **结构化序列生成视角**：动作是 token 序列，但通过“<tool_call> JSON”结构让它看起来像动作；ToolRL、Tool-Zero 等属于此类。 [76]
- **token-policy RL 视角**：形式上是 $\pi_\theta(y \mid x)$ 的 RL（GRPO/GSPO/SAPO），但 token 是动作；奖励通常仍围绕“答案正确/调用匹配/格式正确”。 [77]
- **过程奖励/分层信号视角**：TRM、AgentPRM 把 reward 从终局向过程扩展，显式讨论粒度错配与信用分配。 [78]
- **潜在动作/潜在策略视角**：CoLA、LSPO 把决策从 token 空间提升到 z 空间（latent actions/latent strategies），并用 RL 或博弈求解优化 z-policy，再把语言作为实现层。 [57]

- **“状态/可行动作集”学习视角**：DTDR、TheMCPCompany 把重点放在“工具发现/检索/依赖子图”，相当于学习 $\omega(q, f_{0:t-1})$ 或构建可行动作集。 [79]
- **真实复杂度评测视角**：WILDAGTEVAL 把现实 API 世界的复杂性（规范与执行）显式化，从而暴露训练目标与真实环境之间的结构错配。 [5]

跨这些类别，一个高频的“共同失配（mismatch）”是：**方法宣称在学 policy，但动作建模仍停留在 token 或标签，而环境转移与工具集合变化并未被内生化**。WILDAGTEVAL 与 TheMCPCompany 把“复杂规范 + 执行噪声 + 巨大工具空间”的不可忽略性讲得非常清楚。 [18]

工具使用建模的分类法与核心缺口

分类法：从模式模仿到策略学习

依据“决策变量在哪里”和“优化信号是否来自环境回报”，可把 2025–2026 方法划分为四类（从弱到强）：

模式模仿（pattern imitation / schema matching）：Hammer 代表。其本质是对齐标签/格式、增强描述依赖，解决的是输入表面形态变化（重命名）而非决策。 [80]

结构化生成（structured generation）：ToolRL、Tool-Zero、Tool-R0（Solver 部分）等。输出是结构化 JSON，看似“动作”，但学习信号高度绑定参考调用或可验证语法规则。 [81]

潜在动作建模（latent action modeling）：CoLA、LSPO，外加 ARISE 把“技能库”内生为状态并用层级策略因子分解。它们把决策变量提升到 z 或“技能选择”，更接近“正确动作空间”的方向。 [82]

策略学习（policy learning, environment-grounded）：严格来说，2025–2026 在工具调用上还**没有完全到位**。最接近的是 TRM/AgentPRM 这类把过程奖励与 RL 更新耦合的方法，以及 Tool-R0 这类把任务分布生成也纳入优化的自博弈框架；但它们仍主要在“可验证结构世界”中训练，而非 WILDAGTEVAL 所刻画的真实 API 噪声与规范不完备世界。 [83]

核心缺口：当前方法缺的到底是什么？

综合上述文献，最核心缺口可以表述为：

缺口一：动作语义与工具身份未解耦（Tool Invariance 仍是外生的）
Hammer 用 masking 强迫模型不看名字，属于数据增强；DTDR 用检索子图减少干扰；但这些并没有学习一个可迁移的“工具语义表示”与“语义到可执行调用”的 grounding 映射。 [84]

缺口二：环境转移与执行噪声没有进入训练目标
WILDAGTEVAL 指出真实部署要处理：ad-hoc rules、功能边界不清、依赖关系、执行噪声、部分失败、系统失败等；而多数训练奖励仍在对齐参考调用或可解析结构。 [85]

缺口三：长程信用分配与“何时调用/是否调用/调用后如何修正”仍缺统一表述
TRM 把工具调用效用做成过程奖励，显著推进了 credit assignment；但它仍把动作映射回 token，并且 TRM 本身可能成为新的泛化瓶颈。 [86]

缺口四：开放集工具空间下的策略学习结构仍缺少“可优化且表达强”的统一设计
TheMCPCompany 显示在 18k+ 工具下，oracle 与检索式 agent 仍有明显 gap；这说明仅靠提示或静态检索无法解决“巨大工具空间下的决策”。GoRL 给出了一个跨领域的结构原则：把可优化 latent policy 与表达性 decoder 解耦；但该原则还没有被系统地移植到工具调用。 [87]

候选方向的批判性评估（A–D）

潜在动作 + Grounding（A）

核心思想：学习一个“意图/操作原语”潜变量 z，并学习把 z grounded 到具体工具调用 a 的映射：

$$\pi_\theta(a \mid s, \mathcal{T}) = \int \pi_{\text{hi}}(z \mid s)\, \pi_{\text{grd}}(a \mid s, \mathcal{T}, z)\, dz$$

其中 π_{hi} 做决策（选“做什么”），π_{grd} 做可执行实现（选“用哪个工具、怎么填参、怎么串联输出依赖”）。这一思路在 CoLA（latent actions）、LSPO（latent strategies）得到强支持；ARISE 更进一步把“技能库”内生为状态，并给出层级策略分解与端到端梯度塑形的动机。[82]

优势：
- 直接对齐“工具不变性”：若 z 表示语义操作（如“查订单状态/提取关键信息/格式化参数”），重命名工具或替换 API 只影响 grounding 层。Hammer 用 masking 只能外生逼迫这种效果，而 latent 层提供了结构性路径。[88]
- 可扩展到巨大工具集合：π_{hi} 维度受控，π_{grd} 接收 $\mathcal{T}$ 并做检索/约束解码，可与 DTDR 类模块融合。[79]

劣势/风险：
- z 的可辨识性与训练信号：如果没有可靠的“意图监督”或可验证环境回报，z 可能退化为另一种隐式模板。LSPO 通过聚类 + CFR + DPO 规避了一部分，但工具世界更开放、语义更细碎。[89]
- grounding 需要硬约束（schema、依赖、执行异常）：WILDAGTEVAL 的复杂规范与噪声表明 grounding 不只是生成 JSON。[5]

可行性判断：高。因为已有可借鉴的 latent-policy 框架（CoLA/LSPO/ARISE）与可验证工具输出结构（ToolRL/TRM）可提供训练信号接口。[90]

轨迹级策略优化（B）

核心思想：把工具调用当作真正的多步轨迹优化：

$$\max_\theta \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_t \gamma^t r(s_t, a_t)\Big]$$

并把 reward 与评测对齐到“执行后任务成功/成本/鲁棒性”。

2025–2026 的证据链：TRM 通过 turn-level reward + outcome reward，明确把 credit assignment 从终局拉回到每次工具调用；AgentPRM 把 process reward model 作为 critic-like 信号，专门针对 agent 场景稀疏回报；Tool-R0 用自博弈生成训练分布并用 GRPO 迭代优化；这些都在把学习信号向“轨迹”靠拢。[91]

优势：
- 与真实部署一致：WILDAGTEVAL 说明失败来自执行噪声与规范复杂度，轨迹级优化才能迫使策略学会“失败分支上的再规划与自修复”。[92]
- 可自然纳入成本/延迟：TheMCPCompany 把“步骤数与推理成本”作为关键指标，轨迹级目标能直接优化这些 tradeoff。[49]

劣势/风险：
- sample complexity 与 reward hacking：TRM 明确展示 naive advantage 聚合会导致“少用工具”这种 hacking，需要更细粒度的 advantage 设计；Scaf-GRPO 也展示“零奖励不可见”的普遍性。[93]
- 实际环境难以大规模 rollout：需要可验证的 proxy（如执行日志、程序化验证、规则检查、或强 judge），而这些 proxy 本身可能偏置策略。[94]

可行性判断：中高。若能结合“可验证中间信号”（TRM/PRM）与更稳定的序列级优化器（GSPO/SAPO），轨迹级优化可控性显著提升。[95]

生成式策略（diffusion / flow）（C）

核心思想：把动作分布建模为生成模型（多峰、多模态），用 diffusion/flow 等表示；关键挑战是在线 RL 中 likelihood/梯度不可 tractable。

2025–2026 的结构性进展：
- GoRL 把“优化”限制在 tractable latent policy，把“生成”交给条件生成 decoder，并用两时间尺度更新维持稳定。[96]
- FPO 在 policy gradient 框架内用 flow matching loss 构造近似 ratio 并与 PPO-clip 兼容。[73]
- AEPO 在能量引导 diffusion 的离线 RL 中讨论中间能量估计可计算性。[74]

优势：
- 表达复杂策略分布：工具调用与计划通常是多模态的（多个合理工具链/多个参数化方案），生成式策略更自然。[97]
- 与“潜在动作 + grounding”高度兼容：可以把 tool-call 的参数/计划视为结构化样本，通过生成器实现“可执行性约束下的多样化”。[98]

劣势/风险：
- 离散结构（JSON、工具链图）的生成很难直接套用连续 diffusion；需要把结构映射到连续 latent 后再解码，这又回到 (A) 的问题。[99]
- 训练与推理成本：tool agent 场景强调延迟与成本（TheMCPCompany），生成式采样链可能不可接受，除非采用 GoRL 式解耦与快速解码。[87]

可行性判断：中。更像是 2026–2027 的“加速器”，而不是最先把问题表述做对的关键一步；但 GoRL 的“解耦原则”非常可能成为工具策略学习结构设计的核心部件。[96]

工具不变性 / 表征学习（D）

核心思想：学习与工具名称/命名风格无关的工具表征，使策略对“重命名、参数微调、版本变化、工具扩展”稳健。

证据链：Hammer 用 function/arg masking 强行让模型依赖描述；DTDR 用 history-aware 子图检索减少无关工具的干扰；TheMCPCompany 显示工具发现是核心瓶颈，oracle vs retrieval 的 gap 表明仅提示/检索还不够。[100]

优势：
- 对现实最关键：WILDAGTEVAL 的 ad-hoc 规则与 ambiguous descriptions 本质是工具规范的非平稳性，工具不变性是必要条件。[5]
- 工程收益立竿见影：masking 可直接提升跨命名风格鲁棒性。[12]

劣势/风险：
- 单独做表征学习不足以产生“策略”：它解决的是输入/动作符号映射的稳定性，不自动解决多步决策与信用分配。[101]
- 容易退化为数据增强：没有把不变性作为优化目标的一部分（例如对等价重命名的对比约束、或把工具表示嵌入策略梯度路径），就很难支撑开放世界泛化。[102]

可行性判断：高，但必须与 (A)/(B) 结合，才能从“鲁棒识别”走向“鲁棒决策”。[103]

最有前景的 2026 级别问题表述与可检验假设

最有前景的表述：面向开放工具集合的“潜在策略 + 可执行 Grounding”的层级（POMDP）策略学习

综合 2025–2026 的证据，我认为最有潜力在 2026 做到 NeurIPS/ICML/ICLR 级贡献的“正确表述”应满足三条硬约束：

1) **动作语义与工具身份解耦**：决策变量不应是工具名/JSON 片段，而应是语义操作 z（latent action / skill），参考 CoLA、LSPO 的“潜在动作/策略空间”，以及 ARISE 把技能库内生为状态、用层级策略分解并让梯度影响 skill selection 的思路。[82]

2) **grounding 必须显式条件于工具集合 $\mathcal{T}_t$**：面对 TheMCPCompany 级别的 18k+ 工具，策略必须把工具看成可变集合输入，并在其中检索/约束生成可执行调用；DTDR 提供了一个可操作的 $\omega(q, f_{0:t-1})$ 形式化作为“可行动作集构造器”。[50]

3) **训练信号必须覆盖过程信用分配与执行复杂度**：WILDAGTEVAL 显示 execution noise 与规范复杂度是主要失败来源；TRM/AgentPRM 说明过程奖励是解决稀疏回报与 gradient conflict 的关键；GSPO/SAPO 提供更稳定的策略优化基座。[104]

把这三条合在一起，一个更“正确”的形式化是（给出一种具体而可实现的数学对象）：

- 环境：开放集工具 POMDP，状态/观测 o_t 包含对话、工具返回（含噪声/失败）、以及工具集合表示 $\mathcal{T}_t$。WILDAGTEVAL 提供了复杂度维度来实例化这种转移与观测噪声。[8]
- 层级策略（两层/三层皆可）：

$$\pi_\theta(a_t \mid o_{\le t}, \mathcal{T}_t) = \int \pi_{\mathrm{hi},\theta}(z_t \mid o_{\le t}, \mathcal{M}_t)\, \pi_{\mathrm{grd},\theta}(a_t \mid z_t, o_{\le t}, \mathcal{T}_t)\, dz_t$$

  其中 $\mathcal{M}_t$ 可选为“技能/记忆库状态”，对应 ARISE 的 ES-MDP 思路（把库当作状态内生变量并与策略共同进化）。[105]
- 奖励：三层结构的可验证代理
- r_{exec}：执行后是否推进目标（包含错误分支的修复奖励）；
- r_{tool}：每次工具调用效用（TRM 类过程奖励）+格式/可解析性约束；
- r_{cost}：成本/步数/延迟（与 TheMCPCompany 的成本指标一致）。[106]
- 优化：序列级稳定更新（GSPO/SAPO） + 过程优势估计（TRM 的 turn-level advantage 或 AgentPRM 的 critic-like PRM）。[107]

在这个表述里，“工具调用”不再是分类/模板输出，而是**可执行动作的生成与选择**；“tool invariance”不再靠遮蔽技巧，而是由 z 与 π_{grd} 的解耦结构承载；“环境复杂度”通过 WILDAGTEVAL 式的噪声/错误路径进入优化对象。[108]

可检验研究假设（1–2 个，适合 2026 投稿）

假设一：工具使用必须被建模为“潜在语义动作 + 条件 Grounding”，否则难以获得真正的工具不变性与开放集泛化
- 可操作定义：在训练时对同一任务构造多种“等价工具系统”（工具重命名、参数字段重排、同义描述改写、工具集合增删），要求策略在执行回报上保持不变；比较
1) token-policy + SFT/规则奖励（ToolRL 类），
2) masking 增强（Hammer 类），
3) 显式 latent policy z + grounding（CoLA/LSPO 思路迁移）。[109]
- 预测：只有第 (3) 类能在“工具重命名 + 新工具插入（开放集）+ 执行噪声”三者叠加时保持性能曲线，且对 TheMCPCompany 级巨大工具集合更稳健。[110]

假设二：要在真实 API 复杂度下实现策略学习，必须使用“turn/step 级优势估计 + 序列级稳定优化器”，否则会出现系统性 reward hacking 与学习悬崖
- 证据：TRM 直接展示 group-level advantage 会诱发“少调用工具”的 hacking，提出 turn-level advantage；Scaf-GRPO 指出零奖励导致学习停滞；GSPO/SAPO 讨论 token-level ratio 的不稳定与 sequence-level / soft gating 的必要性。[111]
- 可操作实验：在 WILDAGTEVAL 的 execution error/noise 场景里，比较
- outcome-only RL（GRPO/GSPO），
- +TRM turn-level reward，
- +Scaffold/课程（Scaf-GRPO/Tool-R0 式 mid-training），
并测量：成功率、工具调用次数、错误分支恢复率、以及对噪声强度的鲁棒性曲线。[112]

这两条假设的共同目标是：把“工具调用”从 token 模式模仿推进到“带可执行语义的策略学习”，并在 2026 的评测标准（开放集工具 + 真实执行复杂度）下得到可复现的提升。[113]

---

[1] [9] [10] [11] [12] [14] [80] [84] [88] [100] [101] [102] [103] [108] https://openreview.net/notes/edits/attachment?id=tXO2sxfKzS&name=pdf
https://openreview.net/notes/edits/attachment?id=tXO2sxfKzS&name=pdf

[2] https://arxiv.org/pdf/2511.16043
https://arxiv.org/pdf/2511.16043

[3] [4] [6] [15] [16] [17] [76] [81] [109] https://arxiv.org/html/2504.13958v1
https://arxiv.org/html/2504.13958v1

[5] [8] [13] [18] [44] [45] [46] [47] [85] [92] [104] [112] [113] https://arxiv.org/pdf/2601.00268
https://arxiv.org/pdf/2601.00268

[7] [30] [31] [32] [33] [34] [35] [36] [78] [83] [86] [91] [93] [94] [95] [106] [107] [111] https://openreview.net/pdf/a20beab64b7ad22a1db952ca220305851ae470f1.pdf
https://openreview.net/pdf/a20beab64b7ad22a1db952ca220305851ae470f1.pdf

[19] [20] [21] [22] https://aclanthology.org/2025.findings-emnlp.485.pdf
https://aclanthology.org/2025.findings-emnlp.485.pdf

[23] [24] [25] [26] [27] [28] [29] https://arxiv.org/pdf/2602.21320
https://arxiv.org/pdf/2602.21320

[37] [38] [39] [40] [41] [42] [43] [79] https://arxiv.org/pdf/2512.17052
https://arxiv.org/pdf/2512.17052

48 49 50 87 110 https://openreview.net/pdf/e7aa07f2514665207e4710a52c9434c75a915084.pdf
https://openreview.net/pdf/e7aa07f2514665207e4710a52c9434c75a915084.pdf

51 52 53 54 55 https://arxiv.org/pdf/2502.10325
https://arxiv.org/pdf/2502.10325

56 57 58 82 90 https://proceedings.mlr.press/v267/jia25e.html
https://proceedings.mlr.press/v267/jia25e.html

59 60 61 63 89 https://arxiv.org/pdf/2502.04686
https://arxiv.org/pdf/2502.04686

62 https://arxiv.org/abs/2502.04686
https://arxiv.org/abs/2502.04686

64 65 77 https://ar5iv.org/pdf/2507.18071
https://ar5iv.org/pdf/2507.18071

66 67 https://ar5iv.org/html/2511.20347v2
https://ar5iv.org/html/2511.20347v2

68 69 https://arxiv.org/pdf/2510.19807
https://arxiv.org/pdf/2510.19807

70 71 https://openreview.net/pdf?id=jGbRWwIidy
https://openreview.net/pdf?id=jGbRWwIidy

72 75 96 97 98 https://ar5iv.org/pdf/2512.02581
https://ar5iv.org/pdf/2512.02581

73 99 https://openreview.net/pdf/376ea41422d8f63f58a4916b9a9783de90e8e461.pdf
https://openreview.net/pdf/376ea41422d8f63f58a4916b9a9783de90e8e461.pdf

74 https://neurips.cc/virtual/2025/poster/116269
https://neurips.cc/virtual/2025/poster/116269

105 https://arxiv.org/pdf/2603.16060
https://arxiv.org/pdf/2603.16060